

- **Asynchronous & Non-blocking I/O** → Ensures high performance and efficient request handling.
- **Event-driven architecture** → Ideal for real-time applications.
- **Rich package ecosystem via npm** → Provides extensive libraries for backend functionalities.

2.3.2 Web Framework with Express.js

A web framework simplifies backend development by managing request handling, authentication, and database interactions.

Express.js is a lightweight framework designed for Node.js that provides essential functionalities for building web applications and APIs efficiently.

Key Features:

- **Middleware support** → Enhances request/response handling.
- **Routing system** → Simplifies API endpoint management.
- **Easy integration** → Works seamlessly with databases, authentication, and third-party services.

2.3.3 Database Management with Sequelize & MySQL

Sequelize is an Object-Relational Mapping (ORM) library that abstracts SQL queries into JavaScript objects, making database operations simpler.

Key Features:

- Supports multiple databases (**MySQL, PostgreSQL, SQLite**).
- Handles complex relationships (**One-to-One, One-to-Many, Many-to-Many**).
- Provides built-in migration and transaction support for structured database management.

2.3.4 Authentication & Security

Request Validation with Express Validator

Ensures incoming API requests contain valid and secure data, sanitizing and validating user input before processing.

Password Hashing with bcrypt

bcrypt is a cryptographic hashing algorithm used to securely store passwords and sensitive data.

How bcrypt Works:

1. **Salting** → Generates a unique random salt to prevent attacks using precomputed hash tables.
2. **Hashing** → Repeatedly applies the bcrypt algorithm, making it resistant to brute-force attacks.
3. **Storage** → Stores only the hashed password, ensuring security.
4. **Verification** → Compares hashed passwords for authentication.

Token-Based Authentication with JWT

JSON Web Token (JWT) enables secure transmission of user session data.

How JWT Works:

1. **User Login** → A JWT is generated with user-related information.
2. **Token Signing** → The JWT is signed using a secret key.
3. **Client Storage** → The token is stored in local storage or cookies.
4. **Token Verification** → Subsequent requests include the token for authentication.

Key Benefits:

- **Stateless Authentication** → No server-side session storage.
- **Compact & Secure** → Efficient transmission with secure signing.
- **Scalability** → Suitable for microservices and distributed systems.

2.3.5 File Handling & Document Processing

File Uploads with Multer

Multer is a middleware used for handling multipart/form-data requests, particularly for file uploads.

How Multer Works:

1. **Storage Configuration** → Supports disk and memory storage options.
2. **File Filtering** → Allows filtering based on extensions, size, or other criteria.
3. **Metadata Handling** → Provides access to file metadata for validation.

PDF Processing and Image Conversion

To facilitate document previewing, the system extracts the first page of uploaded PDFs and converts it into an image.

Technologies Used:

- **PDF-lib** → Loads and manipulates PDF files.
- **PDF-Poppler** → Converts PDFs into images.
- **Sharp** → Optimizes images through resizing, compression, and format conversion.

2.3.6 Email & Notifications

Email Sending with Nodemailer

Nodemailer enables programmatic email sending using various transport methods, including **SMTP**, **OAuth**, and **third-party email services**.

Use Cases:

- Sending password reset links.

Push Notifications with Firebase Cloud Messaging (FCM)

The **Firestore Admin SDK** is used for backend integration with **Firestore Cloud Messaging (FCM)** to send push notifications to users on web and mobile platforms.

2.3.7 Task Scheduling with Node-Cron

Automating periodic tasks is essential for system efficiency. **Node-Cron** is used for scheduling operations like:

Implementation in Lecture Management System:

1. **Transaction Handling** → Ensures consistency when modifying lecture schedules.
2. **Lecture Replacement** → Marks original lectures as replaced and schedules their restoration.
3. **Restoration Scheduling** → Uses Node-Cron to revert the replacement after a defined period.

2.3.8 Data Processing & Analysis

Excel File Handling with XLSX

The **XLSX** library is used for reading and writing Excel files, supporting:

- **Data Importing** → Extracts structured data and inserts it into the database.
- **Data Exporting** → Converts database records into Excel format for reporting.

Automated Translation with Translate-Google

To support multilingual functionality, **Translate-Google** is used to translate key data fields.

Use Cases:

- **Automatic translation of user inputs.**
- **Ensuring stored data consistency across multiple languages.**

2.3.9 Utilities & Development Tools

Several utility libraries improve backend development efficiency and ensure smooth backend operations:

- **dotenv** → Loads environment variables for configuration management.
- **cors** → Enables Cross-Origin Resource Sharing (CORS) for secure frontend-backend communication.
- **nodemon** → Automatically restarts the server during development when code changes.
- **faker & @faker-js/faker** → Generates fake data for testing and database seeding.

express-validator: ensures that incoming API requests contain valid and secure data.

It helps to sanitize and validate data before it is processed or stored

2.3.10 Summary of Used Packages

Package	Purpose	Key Features
Node	Core runtime environment	Asynchronous, event-driven architecture for backend development.
Express	Web framework for API development	Simplifies routing, middleware handling, and API structuring.

Package	Purpose	Key Features
express-validator	Request validation	Validates and sanitizes incoming requests to ensure security.
Sequelize	ORM for database management	Simplifies database queries, supports migrations, and table relationships.
mysql2	MySQL database driver	Enables fast and efficient MySQL database interaction.
Bcrypt	Password hashing	Securely hashes user passwords before storing them.
Jsonwebtoken	Authentication using JWT	Provides token-based authentication for API security.
Nodemailer	Email sending	Supports SMTP and third-party email providers for automated notifications.
firebase-admin	Firebase service management	Enables Firebase push notifications and other cloud functionalities.
Multer	File upload middleware	Handles multi-part file uploads securely.
pdf-lib	PDF generation	Allows creation and modification of PDF documents.
pdf-poppler	PDF conversion	Converts PDFs into image files for previews.
Sharp	Image processing	Resizes, compresses, and converts images efficiently.
node-cron	Task scheduling	Automates recurring tasks like data cleanup and email reports.
Xlsx	Excel file processing	Reads and writes Excel files for reporting and data handling.
translate-google	Google Translate API wrapper	Automates text translation into multiple languages.
Dotenv	Environment variable management	Loads configuration variables from <code>.env</code> files.
Cors	Cross-Origin Resource Sharing	Allows secure communication between frontend and backend.
Nodemon	Development tool	Automatically restarts the server on code changes.
faker & @faker-js/faker	Data generation	Generates fake user data for testing and seeding databases.

2.4 Firebase Cloud Messaging (FCM)

Firebase Cloud Messaging (FCM) is a third-party cloud-based messaging service provided by Google. It enables applications to send and receive push notifications across multiple platforms, including Android, iOS, and Web

FCM plays a crucial role in enhancing user engagement by delivering real-time notifications, even when the app is in the background or closed. It acts as an intermediary service that manages communication between the backend server and the client application, ensuring efficient message delivery based on device availability and network conditions. Additionally, FCM provides message queuing and automatic resending if the target device is offline at the time of message delivery.

2.4.1 Key Features of Firebase Cloud Messaging (FCM)

- Instant Push Notifications → Provides real-time updates to users
- Cross-Platform Support → Works seamlessly across mobile and web applications
- Targeted Messaging → Supports sending notifications to specific users, groups, or topic-based subscribers
- Reliable Message Delivery → Optimizes battery consumption and ensures messages are received even under weak network conditions
- Background & Foreground Notification Handling → Manages notifications differently based on whether the app is active or running in the background
- Custom Data Support → Allows additional payloads in messages to trigger specific actions within the app
- Automatic Resending for Offline Devices → If a device is offline, FCM queues the message and delivers it once the device reconnects to the internet

2.4.2 Backend Integration

To integrate FCM with the backend, the server must be configured to communicate with the FCM service by:

- Authenticating with Firebase → Ensuring the backend has the necessary .permissions to send messages
- Identifying Target Devices → Messages can be sent to individual - .devices (via FCM tokens) , user groups, or topic-based subscribers
- ,Sending Notifications → The server constructs a message with the title - .body, and optional data payload before sending it to FCM for delivery
- Handling Response & Errors → Firebase returns a response indicating - ,whether the message was successfully delivered or if issues occurred (e.g .expired tokens, authorization errors)
- Offline Message Queueing & Resending → If a device is offline, FCM - stores the message and retries delivery once the device reconnects

2.4.3 Frontend Integration

For the frontend, FCM must be integrated into the application to receive and process notifications:

- Registering the Device → When the app is installed, it generates a unique .FCM token, which is sent to the backend for storage and future use
- Handling Notifications → Notifications are received and displayed .differently depending on whether the app is active or in the background
- Requesting User Permissions → On mobile devices, users must grant .notification permissions for the app to receive and display alerts
- Customizing Notification Behavior → Developers can control how notifications appear, whether they trigger in-app actions, or if they should .be stored for later
- Receiving Delayed Notifications → If the device is offline at the time of message delivery, it automatically receives the notification upon reconnecting to the internet

2.4.4 Notification Flow

The notification process follows these steps:

1 Backend Server

- The server prepares the notification, including the message content and recipient details.
- It sends the request to FCM, specifying whether the notification is for an individual user, a group, or topic-based subscribers.

2 FCM Service

- Firebase authenticates and processes the request.
- **Device Online:** The notification is delivered immediately to the device.
- **Device Offline:** The notification is queued and will be retried until the device reconnects to the internet (max retention period of ~28 days). If the message expires (if an expiration time is set), it will not be delivered.

3 Message Routing

- FCM determines the routing based on several factors:
 - **App State:** Foreground, Background, or Terminated.
 - **Platform:** Android, iOS, or Web.
 - **Delivery Priority:** Normal or High.
 - **Network Status:** Whether the device is online or offline.

4 Frontend Application

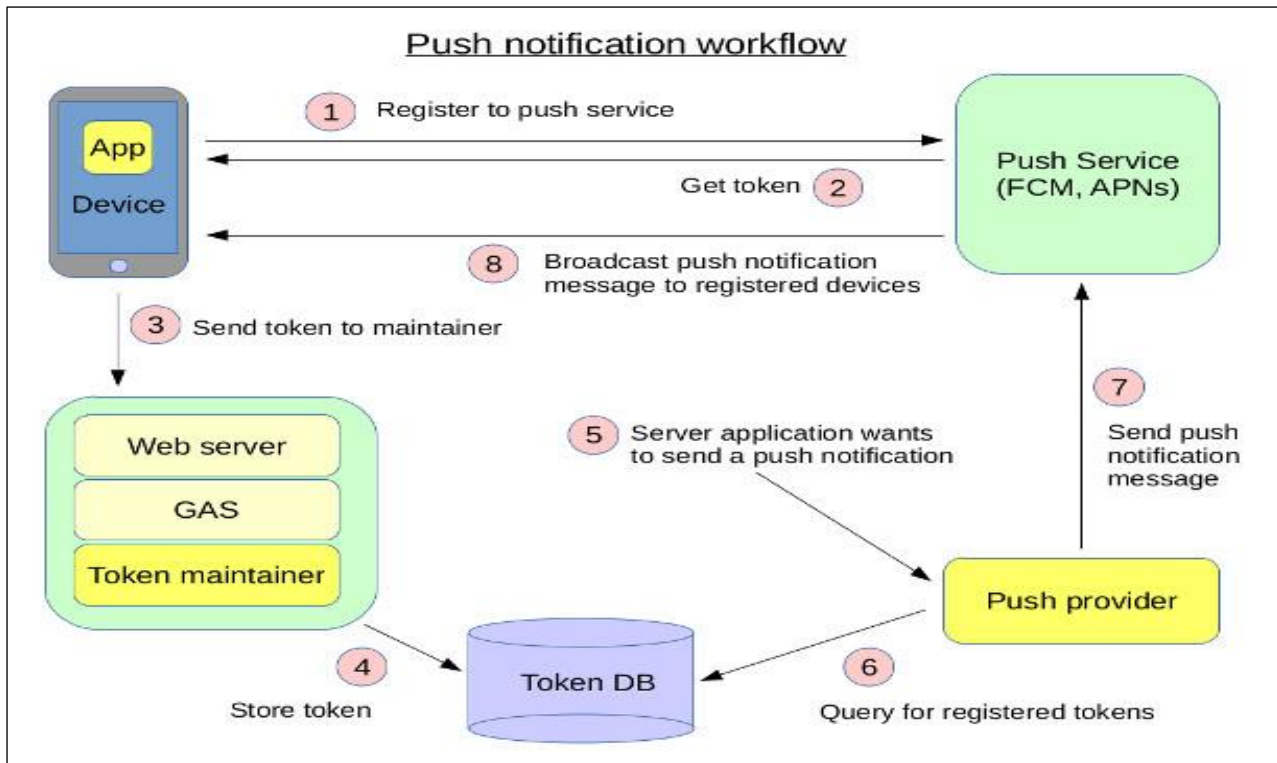
- **App in Foreground:** The app receives the notification instantly, which can trigger in-app actions such as updating the UI or fetching additional data.
- **App in Background or Closed:** The notification appears as a system alert (e.g., in the notification tray).
- **Offline Device:** The notification is queued by FCM and automatically delivered when the device reconnects to the internet.

5 User Interaction & App Response

- **User Taps Notification:** The app is opened, and the message data is passed to the app.
- The app can handle the message, update the UI, and respond accordingly (e.g., fetching more data or navigating to specific content).

2.4.5 Notification Flow Diagram

The following updated diagram illustrates how FCM stores messages when a device is offline and resends them once the device reconnects:



Figuer 2.1 Notification flow

2.5 Development Tools & Utilities

This section outlines the essential tools and utilities used in the development process to ensure smooth collaboration, effective coding, and efficient testing for both frontend and backend. These tools are fundamental for version control, API testing, managing development environments, and handling package dependencies.

2.5.1 Version Control (Git & GitHub)

Version control is crucial for tracking changes in the codebase and for collaboration.

Git and **GitHub** are used extensively in this project to ensure that code changes are tracked efficiently and that developers can collaborate effectively.

Git: A distributed version control system that allows developers to track and manage changes in the source code. It enables:

- Efficient branching and merging of code.
- Collaboration without conflicts, as developers can work on separate branches and later merge their changes into the main branch.
- Tracking of project history and rollback to previous versions if necessary.

GitHub: A cloud-based platform for hosting Git repositories, providing a central location for developers to share and collaborate on code. Key features of GitHub include:

- Pull requests for proposing and reviewing changes.
- Issue tracking to manage tasks, bugs, and feature requests.
- Collaboration tools such as comments, code reviews, and project boards.

2.5.2 API Testing (Postman)

Postman is a comprehensive tool for testing APIs. It is used to ensure the backend services function as expected before integration with the frontend. With Postman, developers can:

- **Send Custom HTTP Requests:** Supports various HTTP methods such as GET, POST, PUT, DELETE, and PATCH, allowing testing of different API endpoints.
- **Verify API Responses:** Developers can verify if the API returns the expected response, including status codes, data, and error messages.
- **Automated Testing:** Postman supports automated testing, enabling the creation of test suites to ensure continuous API reliability.
- **Environment Management:** It allows the management of different environments (e.g., development, staging, production) for seamless testing across different configurations.

2.5.3 Development Environments (VS Code & IntelliJ IDEA)

For efficient coding, debugging, and project management, **VS Code** and **IntelliJ IDEA** are the primary development environments used in this project.

VS Code (Visual Studio Code): A lightweight, versatile code editor that supports many programming languages, including Dart and JavaScript. Features include:

- **Syntax Highlighting:** Makes code more readable and easier to debug.
- **IntelliSense:** Provides code completion, parameter info, quick info, and member lists.
- **Integrated Terminal:** Allows developers to run commands and scripts directly within the editor.

- Debugging Tools: Integrated debugging support for various programming languages, including Dart for Flutter.

IntelliJ IDEA: A powerful integrated development environment (IDE) used for more robust Flutter development. It supports:

- Advanced Code Intelligence: Offers sophisticated code suggestions, completions, and refactoring tools.
- Flutter Integration: Seamless integration with Flutter SDK and its tooling for live UI design and app building.
- Live UI Design Tools: Provides a visual interface for Flutter widget design.
- Robust Debugging Capabilities: IntelliJ IDEA offers comprehensive debugging support, including hot reloads for Flutter development.

2.5.4 Package Management (npm)

npm (Node Package Manager) is the default package manager for the Node.js runtime environment. It is used to manage the dependencies and libraries for the backend (Node.js and Express.js) of the project. Key features of npm include:

- Package Installation: Helps developers install project dependencies and ensures the correct versions of libraries are being used.
- Version Management: npm ensures compatibility between different package versions, avoiding issues related to outdated or incompatible libraries.
- Speeding up Setup: By managing dependencies, npm ensures quick setup and installation processes for new developers or in new environments.

2.5.5 Summary of Development Tools & Utilities

Tool	Purpose	Key Features
Git	Version control system	Tracks code changes, supports branching, merging, and collaboration.
GitHub	Cloud-based Git repository hosting	Facilitates collaboration through pull requests, code reviews, and issue tracking.
Postman	API testing tool	Allows custom HTTP requests, response verification, automated testing, and environment management.
VS Code	Code editor	Syntax highlighting, IntelliSense, integrated terminal, and debugging tools.

Tool	Purpose	Key Features
IntelliJ IDEA	Integrated development environment (IDE)	Advanced code intelligence, Flutter integration, live UI design, and debugging.
Npm	Package management tool	Installs and manages Node.js dependencies, ensuring correct versions and easy setup.

These tools are essential in ensuring smooth development processes, improving team collaboration, and maintaining high code quality. They streamline the workflow, from coding and testing to version control and deployment.

CHAPTER 3

App architecture and implementation

3.1 App Analysis and Design

This section provides a high-level overview of the system's architecture, design patterns, and key features. It explains how the app is structured and why certain design choices were made to ensure scalability, maintainability, and usability.

Note: This section focuses on concepts and design decisions. Technical implementation details, such as how caching, synchronization, authentication, and API interactions are handled, will be covered in later sections.

System Architecture Overview

The system follows a **Client-Server Architecture**, where different client interfaces communicate with the backend through **REST APIs**. This separation ensures:

- **Scalability** – The frontend and backend can evolve independently.
- **Security** – Business logic and sensitive data are managed on the server.
- **Efficiency** – The frontend handles UI and user interactions, while the backend processes data and enforces rules.

System Components

1. Client (Mobile App & Web Page)

- Used by students and doctors to access services.
- Available as both a mobile app and a web-based interface for accessibility.
- Handles user interactions, displays data, and sends API requests to fetch and update information.
- Supports offline access by caching key data.

2. Admin Panel (Web - Limited to Data Management)

- A web-based dashboard used by administrators exclusively for data management (insert, delete, and update records).
- Interacts with the same backend as the client interfaces via REST APIs.
- Does not include service usage features available in the mobile/web client.

3. Server (Backend / API)

- Processes requests, applies business rules, and returns responses.
- Manages authentication and authorization.
- Ensures data consistency and synchronization.

4. Data Storage & Communication

- A relational database (MySQL) stores structured data efficiently.

- The mobile app, web client, and admin panel communicate with the backend via RESTful APIs.

Design Patterns & Principles

Using structured design patterns makes the system maintainable, scalable, and well-organized by separating concerns and reducing complexity.

Key Patterns Used

- **Model-View-Controller (MVC)** – Separates data, UI, and logic, improving maintainability. The Model represents data (e.g., users, services), the View displays it, and the Controller manages interactions.
- **Repository Pattern** – Provides a flexible data layer, making it easier to modify queries or switch databases without affecting business logic.
- **Service Layer Pattern** – Organizes business logic into dedicated services such as authentication, notifications, and data synchronization, keeping controllers lightweight and reusable.
- **Component-Based UI Design** – Encourages reusability in the frontend. Global components like buttons and cards are shared across screens, while view-specific components are customized for each page.

User Experience Considerations

Multi-Language Support

To enhance accessibility, the app supports multiple languages, specifically Arabic and English. Multi-language support is implemented at both the frontend and backend levels:

Frontend

- The UI dynamically switches languages based on the user's preference.
- Translations are stored in separate text resource files.
- Proper **right-to-left (RTL) support** is implemented for Arabic.

Backend

- **Translatable Fields:** The database uses a JSON column to store translations for multi-language fields, allowing easy retrieval of content in different languages.
- **Auto-Translation:** When new records are inserted, the system **automatically** translates content using the Google Translate API, reducing manual effort.

Offline Support

Since users may lose internet access during usage, the app ensures a smooth and uninterrupted experience through offline support mechanisms.

Caching Data

The app uses **local storage (Hive)** to cache essential data such as lectures, assignments, user details, and library files. This ensures that:

- Users can continue accessing key features even without an internet connection
- Previously loaded data is displayed instantly from local cache
- Redundant API requests are avoided, improving performance and efficiency

Caching is closely integrated with the app's repository layer, enabling automatic storage and retrieval of structured model objects.

Data Synchronization Mechanism

To keep cached data accurate and up-to-date, the system implements a custom **timestamp-based synchronization mechanism**. This strategy is designed specifically for this app, blending ideas from traditional delta sync with additional support for filtered data scopes and real-time triggers.

Backend Processing

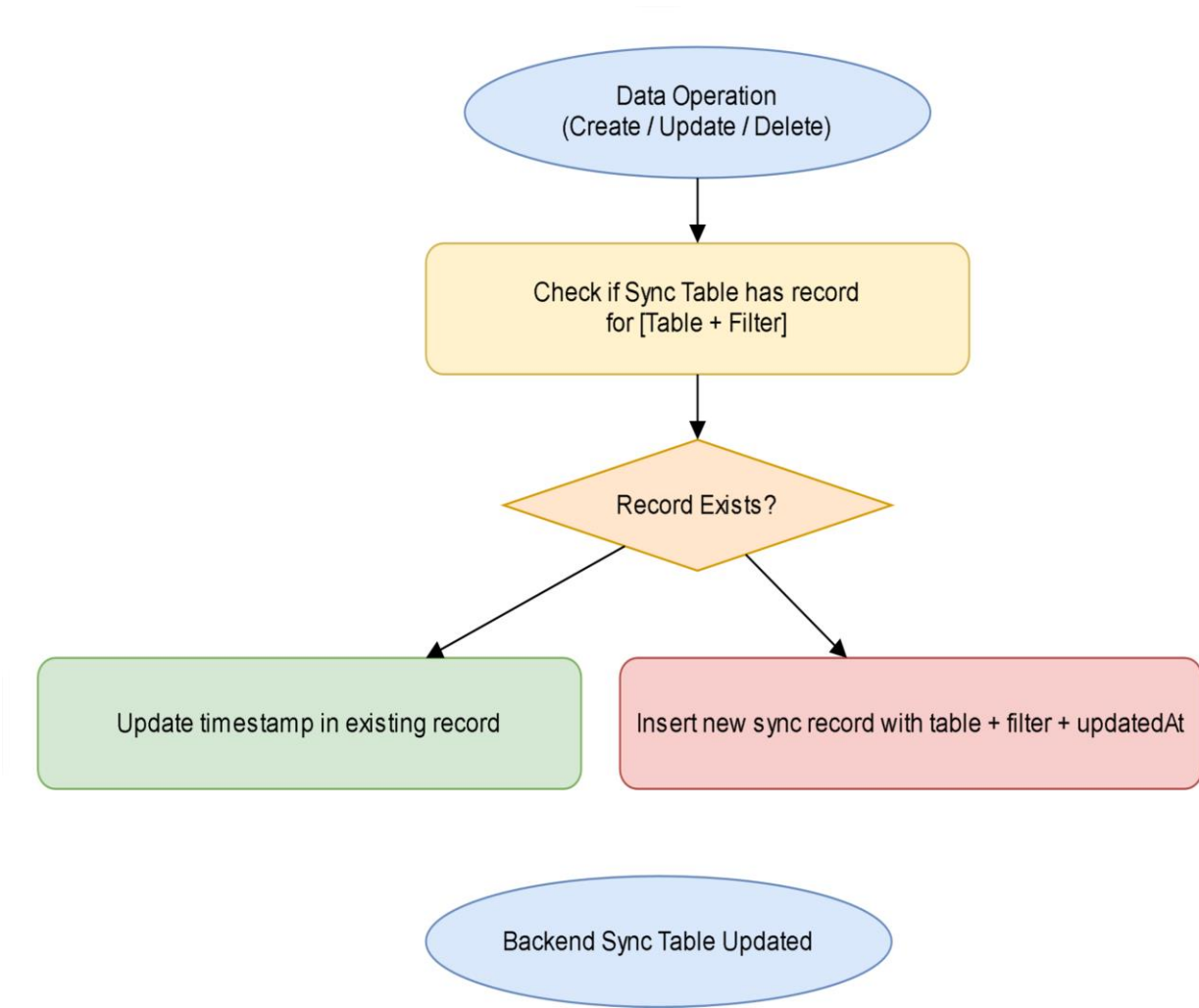
The backend maintains a dedicated **Sync Table** that tracks:

- The last updatedAt timestamp for each table
- Optional **filters stored as JSON** (e.g., level ID, section ID), allowing different states for subsets of data

Whenever a data operation occurs (create, update, delete), the backend either:

- Updates the timestamp of an existing sync record for that table and filter
- Or inserts a new record if no matching state exists

This enables precise detection of what data has changed and under which filter.

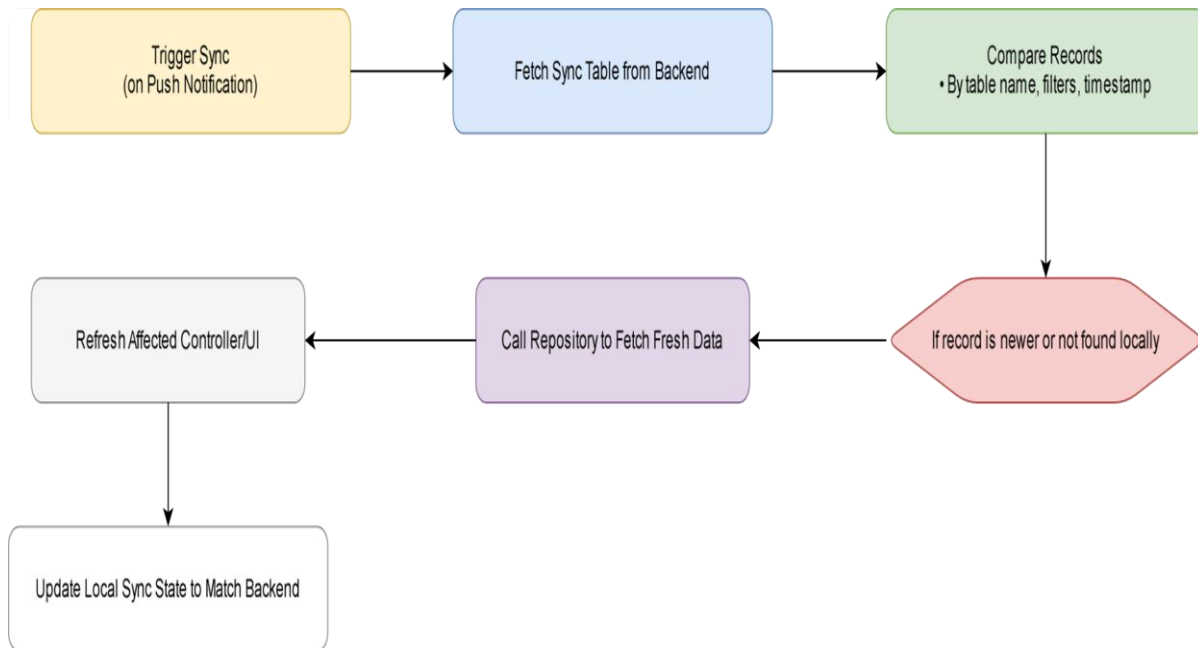


Frontend Synchronization Flow

The frontend keeps a local copy of the last known sync state. The sync process is triggered when a push notification (via FCM) is received

The sync service performs the following steps:

1. **Fetch the latest Sync Table** from the backend
2. For each record, **compare timestamps and filters** against the local copy
3. If a newer timestamp is found (or if the record didn't exist locally), trigger a **targeted data refresh** via the relevant repository
4. After all updates are processed, the local sync table is updated to match the backend



This approach ensures that:

- Cached data stays consistent without requiring full refreshes
- Data sync is context-aware (e.g., syncing only relevant records for the user)
- Users experience a responsive and seamless app — both online and offline

Security & Performance Considerations

To ensure system reliability and protect user data, the app incorporates various security and performance optimizations:

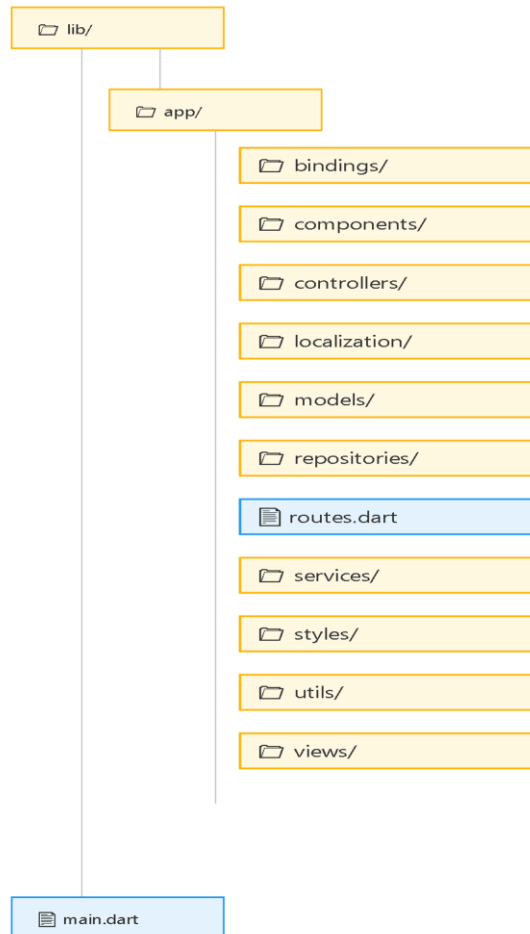
- **JWT Authentication** – Users authenticate using JSON Web Tokens (JWT), which secure API requests.
- **Role-Based Access Control (RBAC)** – User permissions vary depending on their role (e.g., student, doctor, admin).
- **Local Storage Caching** – The frontend caches data locally for faster loading times and offline data view.

3.2 Frontend Implementation

This section presents how the frontend of the app was structured and built using Flutter. It focuses on architectural decisions, tools, and the connection between UI components and logic, without going into full development details.

1. Project Structure

As mentioned earlier, we follow the **MVC (Model-View-Controller)** pattern, which is a great way to keep everything organized. It helps us separate the app's logic, UI components, and data management, making it easier to scale and maintain over time. Let's dive into how the project is structured:



Folder Breakdown: What Each Folder Does

app/

This is where the core of the app resides, containing everything that drives the app's functionality. Here's what each folder in app/ is responsible for:

- **bindings/:**
Handles the **setup and dependency injection** for the app. This ensures each component gets what it needs to function properly.
- **components/:**
Contains **reusable UI elements** that appear across multiple screens, such as buttons, cards, and custom widgets. This promotes consistency and avoids duplication of code.
- **controllers/:**
Houses the **business logic** of the app. These controllers manage the app's

state, fetch data from the backend, and update the UI as necessary. We use **GetX** for reactive state management, which ensures the UI updates automatically when the data changes.

- **localization/:**
Manages **language support**, enabling the app to be used in different languages.

It contains translation files and Getx localization setup class

- **models/:**
Defines the **data structures** used throughout the app. These models represent the data (e.g., user details, lecture information) and allow us to easily parse and manipulate it.
- **repositories/:**
Responsible for **data fetching** and **caching**. Repositories handle the app's communication with APIs or databases and cache the data for future use.
- **services/:**
Contains code for integrating with **external services** like APIs, Firebase, and storage solutions. These services help the app communicate with the outside world.
- **styles/:**
Holds the app's **styling and themes**, such as colors, fonts, and layout settings, ensuring a consistent look and feel throughout the app.
- **utils/:**
Contains **helper functions** for common tasks, like formatting dates or validating input. These functions are used across the app to simplify repetitive code.
- **views/:**
Houses the **UI screens** for different pages of the app. Each screen is defined here, where we specify how the UI should be displayed, including layouts, components, and user interactions.
- **routes.dart:**
Manages **app navigation** by defining named routes. It makes navigating between screens easier and more consistent.

main.dart

This is the **entry point** of the app. It initializes the app's configuration (like themes, routes, and services) before the app starts running.

Key Principles Behind This Structure

- **Separation of Concerns:** We keep the UI, business logic, and data management in separate areas to make the app easier to work with. This structure is clean and ensures that developers can focus on one part of the app at a time without getting overwhelmed.

- **Feature-Based Structure:** Each feature (like login, profile management, or lectures) has its own dedicated folder, which makes it easy to scale the app as new features are added.
- **Reusability:** By creating reusable components, we can avoid repeating code. If something is used on multiple screens, we put it in the components folder and just reference it when needed.

2. GetX Role in Frontend Implementation

As previously mentioned in **Chapter 2: Technology and Tools**, the app uses **GetX** as the main state management and routing solution due to its lightweight nature, reactive programming support, and built-in tools for navigation, dependency injection, and localization.

In this chapter, we focus on how GetX was applied practically throughout the frontend implementation.

GetX played a key role in multiple layers of the app, including:

- State and logic management through reactive controllers
- Modular screen routing
- Centralized service injection
- UI reactivity and responsiveness
- Localization handling
- Utility access for screen sizing and messaging

These features contributed to building a clean, organized, and scalable frontend structure.

Core GetX Components Used in the App

Below are the key GetX components integrated into the app, along with their purpose and how they contribute to the app:

App Foundation

- **GetMaterialApp**
Acts as the backbone of the entire app. It replaces the regular **MaterialApp** and enables all of GetX's features — routing, state management, localization, snackbars, dialogs, and more — with zero extra setup.

Used in: `main.dart` as the root widget to initialize the app.

Controllers and State Management

- **GetxController**
Every major screen has a dedicated controller that manages its state and logic, such as handling API calls, updating values, and reacting to user actions.

Example: LoginController manages input validation and calls UserRepository.login() when the login button is pressed.

- **GetView <T>**
A convenience class used in screen widgets to easily access their controller without writing extra boilerplate code.
- **Obx**
The most frequently used widget for observing reactive (Rx) variables. Any change in data automatically updates the UI without manual triggers.
- **GetBuilder**
Used in a few places where performance matters, allowing manual rebuilds using update(). This is helpful for situations where reactivity doesn't need to be continuous.

Dependency Injection and Modularity

- **Bindings**
Each screen has an associated binding class, responsible for injecting its dependencies. This keeps screen setup modular and clean, especially for large apps.
- **Get.put () / Get.lazyPut()**
Used to inject controllers and services into memory:

- Get.put () immediately creates and stores the instance.
- Get.lazyPut() delays the creation until it's first used.

Controllers are usually registered through bindings using lazyPut () for efficiency.

Routing and Navigation

- **GetPage**
Used to declare all the app's named routes. It connects each screen with its binding and helps in maintaining centralized route management.
- **Navigation methods: Get.to (), Get.off ()**
These methods simplify navigating between screens, without needing to use context.
 - Get.to () → push a new page
 - Get.off () → replace the current page

Used in flows like login → home, or navigating from subject list to details.

Localization & Language Handling

- **.tr and Get.updateLocale()**
GetX provides built-in support for internationalization. All display strings are stored in translation maps and accessed using (.tr).
The app supports Arabic and English, including proper RTL layout support.

Feedback and UI Helpers

- **Get.snackbar()**
Used to give quick feedback to users for actions like login success, form errors, or saving data.
- **Get.dialog()**
Used to display custom popup cards, such as forms for adding or editing data, without needing context. It simplifies dialog handling and helps keep UI interactions clean and consistent.

Example: When a user taps the “Add” button, a styled popup card appears using Get.dialog() to collect input.

- **Get.width / Get.height**
Helpful utilities to get screen dimensions directly, without Media Query, allowing cleaner responsive layouts.

This modular and consistent use of GetX helped simplify logic, reduce boilerplate, and maintain a clean separation between UI and business logic throughout the app.

3. Service & Utility Classes

To maintain a clean and modular architecture, the app uses a set of service and utility classes. These classes handle core functionality such as API communication, authentication, notifications, and localization—keeping logic out of UI components and controllers for better separation of concerns.

This section outlines the main service and utility classes used throughout the app, explaining their responsibilities and how they contribute to the overall structure.

○ HTTP Provider class

The `HttpProvider` class acts as the core communication layer between the app and the backend API. It's built on top of Dio, a powerful HTTP client for Dart, and provides a centralized, consistent approach to handle all HTTP requests across the app.

Key Responsibilities:

- Manages all standard HTTP methods: GET, POST, PUT, and DELETE.
- attaches headers to requests (e.g., JWT authentication tokens).
- Stores and updates the authentication token securely.
- Handles file uploads and downloads, including multipart/form-data and streaming handling.
- Centralizes error handling by catching exceptions and returning consistent responses.

This design helps isolate networking logic, making it reusable, testable, and easy to maintain, while giving calling layers full control over how they process the received data.

○ Notification Service class

The notification service integrates with Firebase Cloud Messaging (FCM) to manage push notifications.

Responsibilities:

- Initializes FCM and listens for new messages.
- Displays notifications while the app is in the foreground.
- Handles redirection or actions based on notification data.
- Prepares the app for background and terminated-state message handling.

This ensures real-time updates and enhances communication with users.

○ HiveServices Classes

This class manages local data storage using the Hive database. It registers all required adapters for custom models and handles opening, closing, and clearing Hive boxes.

Key Responsibilities:

- Registers all model adapters used across the app.
- Opens global data boxes on app startup (e.g., lectures, subjects, users).
- Provides methods to clear or close boxes when needed.

It plays a key role in enabling offline access by storing frequently used data locally.

- **DataSyncService class**

The DataSyncService class is a central utility responsible for handling **frontend data synchronization** when changes are detected on the backend. It ensures that only updated data is re-fetched from the server and that the app's cache stays consistent.

This class works closely with the backend's sync tracking mechanism and plays a key role in the app's offline support and performance optimization.

Key Responsibilities

- Compare the current sync state with the last locally cached version.
- Refresh data only if newer updates exist.
- Trigger the appropriate repository methods to refetch and update local cache.
- Notify relevant UI controllers (if active) to refresh their state after sync.

4. Utility Classes

A number of utility classes are defined to support common tasks across the application. These are small, focused classes that encapsulate specific helpers and tools.

Below is a description of each:

1. `double_digits_parse.dart`

This utility ensures that floating-point numbers are displayed with a consistent number of digits, often formatting values to include a leading zero or fixed decimal places when necessary.

It's especially useful for creating uniform visual representations in areas like scores, prices, or measurements.

2. `internet_connection_checker.dart`

Handles internet availability detection, helping the app respond to online/offline states (e.g., show a message, block a request, cache offline data).

3. `local_lisenter.dart`

wrap app local on observable variable and handle Local update globally.

4. responsivity.dart

Currently includes a single but important function: `fontScale ()`, which adjusts text sizes based on screen dimensions or scaling logic — helps maintain visual consistency across devices.

5. snake_bar.dart

WrapsGetX's `Get.snackbar()` functionality in a cleaner helper function for uniform usage throughout the app. Likely offers styling presets or reusable logic (e.g., success/error messages).

6. date_time_utils.dart

Formats and parses date/time strings, durations, timestamps — typically used in UI and reporting.

7. file_utils.dart

Manages file-related tasks such as opening, writing, deleting, files.

8. json_utils.dart

Handles safe JSON parsing for encoding json content.

9. mapping_data.dart

Facilitates the transformation or mapping of raw, relational, or numeric data into a display-friendly format. This is especially useful when backend data needs to be converted into human-readable forms, such as mapping status codes, category IDs, into user-friendly values being view in screen.

10. screen_utils.dart

Provides screen type determines function to get target device view (web or mobile).

11. validators.dart

A set of reusable input validation methods (e.g., email format, required field, password strength), used in forms across the app.

5. Authentication & Authorization on Client Side (Front-End)

On the client side, authentication and authorization are key for securing access to different parts of the app. The app uses JWT (JSON Web Token) for authentication and role-based for authorization.

1. Authentication:

1. **JWT Token:** Upon successful login, the backend issues a JWT token, which is securely stored in HiveBox.
2. **Token Validation:** Every time the app makes an API call, the JWT token is included in the request headers. The server validates the token before processing the request.
3. **Logout:** To log out, the token is removed from HiveBox, and the user is redirected to the login screen.

2. Authorization:

1. **Role-based Access:** The app uses role-based authorization to manage user permissions. The backend sends the user's role (e.g., admin, student) along with the user data.
2. **Front-End Role Handling:** Depending on the user's role, the front-end will show or hide certain elements or features (e.g., only admins can access certain settings like add).
3. **Authorization Check:** The front-end checks for the user's role before view screens. If the role is unauthorized some elements will hide like add button

6. Repositories and Data Access Layer

Repositories are responsible for **organizing and abstracting all data-related logic** in the app. They act as a bridge between the backend API, local storage, and the controllers.

Each feature module (such as lectures, users, or assignments) has a corresponding repository that centralizes all operations for that type of data.

Key Responsibilities

- Handle **retrieval, creation, updating, and deletion** of data
- Serve as the single access point for each data type (e.g., subjects, exams)
- Support both **online API communication** and **offline Hive caching**
- **Automatically cache** responses after fetching from the server
- **Automatically check cache** before making a request (unless forceFetch is true)
- Simplify how the rest of the app accesses data (controllers don't need to handle cache or fetch logic manually)

7. Models and Data Structure

Models are the backbone of the app's data structure on the frontend. Each backend entity has a corresponding Dart class that defines how the data is represented, used, and passed around within the app.

Key Responsibilities

- **Structure:** Models clearly define the shape and fields of each data entity.
- **Hive Adaptivity:** Models are annotated with Hive type adapters, allowing them to be saved and retrieved directly using Hive's local NoSQL storage.
- **Consistency:** They ensure a consistent, typed structure when sending and receiving data from the backend, enabling safer and more readable code.
- **Reusability:** Models are reused across views, controllers, and repositories for both API parsing and local caching.

in this app architecture, repositories handle the actual saving and fetching of model objects, while models define the schema that represent data on the app and allows Hive to serialize/deserialize the data automatically.

Backend-to-Frontend Mapping

For every main entity on the backend (e.g., Student, Subject, Assignment), there is a **corresponding model** on the frontend. This 1:1 mapping ensures tight synchronization between frontend and backend structures, making data handling easier and more predictable.

Multi-language Fields

Some models contain fields that need to support multiple languages. Instead of duplicating fields for every language, these models use a **map-based structure**:

```
@HiveField(1)
Map<String, dynamic>? nameData;
```

A custom **getter** is used to fetch the correct value based on the currently selected language:

```
String? get name {
  String currentLang = Get.locale?.languageCode.toString()??"en";
  return nameData?[currentLang];
}
```

This keeps the models compact and easily adaptable to language changes without additional transformation layers.

8. Controller Structure & Usage in Screens

Controllers in the app serve as the **central layer for managing screen-specific logic and reactive state**. Each controller acts as the **logic brain** behind a screen or module, keeping UI code clean and separating concerns.

The app uses **GetX's** `GetxController` as the base for all controllers, which provides lifecycle hooks, reactivity, and tight integration with bindings and services.

Key Responsibilities of Controllers

- Manage screen-level data and state (e.g., form input, loading indicators)
- Validate user input and interact with form elements
- Connect to repositories or services for data operations
- Trigger navigation and localization changes
- Expose Rx variables that the UI observes using `Obx` or `GetBuilder`
- Optionally provide dynamic UI elements like dropdown options or labels used across widgets

While controllers do not contain full UI layouts, they **may include UI-related elements** that need to be controlled or updated dynamically (e.g., list of `DropdownMenuItems`, tabs, chips, etc.).

How Controllers Are Integrated

Controllers are tightly integrated into the app through **GetX's binding system**, which manages dependency injection and lifecycle behavior automatically.

- **Created per screen or feature module** to keep logic self-contained and modular.
- **Registered through bindings** using both `Get.put()` and `Get.lazyPut()`, depending on the screen's usage pattern:
 - `Get.lazyPut()` is used for **tab-based screens**, where the same controller may be revisited multiple times through navigation. This approach delays initialization until the controller is first used, improving efficiency for screens that remain in memory.
 - `Get.put()` is used for **standalone screens with separate routes**, such as login or detail pages that are accessed through navigation. This ensures the controller is fully initialized when the screen is pushed.
- Accessed in views via:
 - `GetView<T>` — which simplifies direct access to the controller inside a widget class.
 - `Get.find<T>()` — used for manual retrieval when needed.
- Fully **lifecycle-aware**, using `onInit()`, `onReady()`, and `onClose()` to:
 - Initialize state or load data

- Perform logic once the UI is ready
- Dispose resources like TextEditingControllers or FocusNodes

This integration allows the app to remain clean and reactive, while maintaining strong separation between **UI** and **logic**, and providing fine-grained control over controller instantiation and performance.

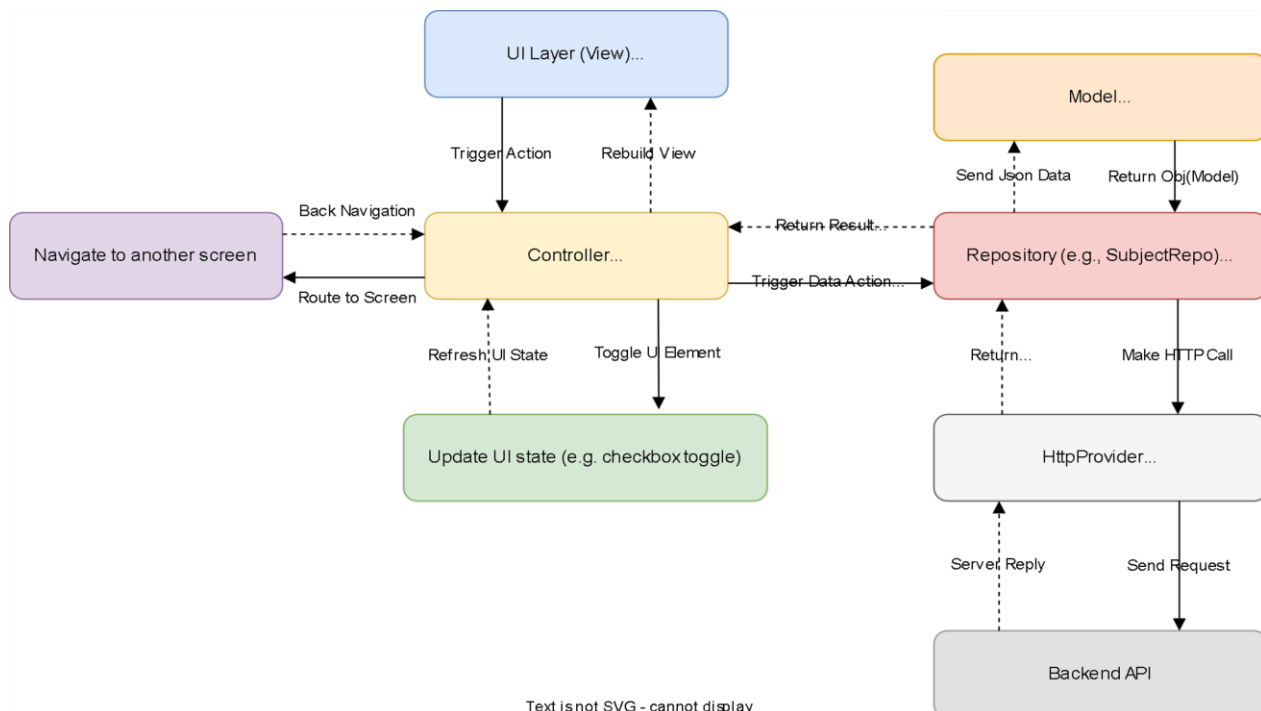
9. Frontend Data Flow and Communication

This section presents a generalized view of how the frontend layers communicate and coordinate during any action — whether it's navigating, toggling a setting, or accessing backend data.

Rather than focusing on specific business logic, this overview shows how **responsibilities are distributed** and how data or instructions flow between components.

Layered Process Flow Overview

The following diagram shows the communication and decision flow between frontend layers. It applies to all user-triggered events, regardless of the specific screen or feature.



Description of the Flow (Narrative Form)

1. A **user interacts with the UI** — for example, taps a button or submits a form.
2. The **controller receives the event**:
 - If it's a **local action** (e.g., checkbox toggle), it directly updates state.
 - If it's a **navigation** event, it triggers route change (e.g., `Get.toNamed()`).
 - If it's a **data access** event, it delegates the work to the corresponding repository.
3. The **repository function executes**, often cross-bonded to a backend entity.
 - It prepares the correct API URL and method.
 - Passes them to the `HttpProvider` to handle the request.
4. `HttpProvider` makes the request:
 - Adds headers and token
 - Waits for the response
 - Wraps the result in a standard `Result` object
5. The **repository receives the response**:
 - Parses it into the appropriate model(s)
 - Caches it locally (Hive) if applicable
 - Returns the result (success or failure) to the controller
6. The **controller updates UI state** (e.g., `RxList` or `RxBool`)
7. The **UI rebuilds automatically** in response to updated reactive variables via `Obx ()` or `GetBuilder ()`

10. UI & Screens (View) and User Interaction

The View Layer is the part of the application responsible for presenting data and receiving user input. It is located under the `app/views/` directory and structured around feature-based modules. Views are built using a **component-based architecture** to ensure reusability and visual consistency.

Each screen is linked to its logic using `GetView<T>`, a `GetX` utility that binds the view to its controller. Reactive UI updates are managed through `Obx ()` and `Rx` variables inside each controller. Views do not contain business logic — instead, they simply display the interface and call methods from the controller when interaction occurs.

View Layer Structure

- Views extend `GetView<Controller>` and directly access controller logic without boilerplate.
- Reactive state changes (e.g., loading, toggles) are handled using `Obx ()` widgets.
- Layouts follow a **clean separation of presentation and logic**, using component wrappers and utility classes.